

TechPixel Agent Base44 Training Document

A complete reference for configuring and training the TechPixel content-validation agent on Base44. This document covers every aspect of the agent's domain knowledge: the Bloxd.io game platform (mechanics, gamemodes, scripting APIs, world building) and the BloxdHub.com community platform (posts, mods, schematics, forums, texture packs). It also provides Base44-specific configuration instructions, validation logic, example training notes, and edge-case guidance.

30+	64	7	6
Gamemodes Active on Bloxd.io	World Code Callbacks Event-driven scripting	BloxdHub Sections Mods, Schematics, Forums...	Base44 Config Tabs Guidelines through Telegram

I. Base44 Agent Configuration

1. What Base44 Agents Are

AI agents on Base44 turn chat into action. They understand your app's data, can search the web, call backend functions, and follow detailed guidelines you define. Each agent has its own configuration panel with six core tabs: Guidelines, Tools, Connectors, Skills, Memory, and messaging channels (WhatsApp, Telegram, LINE).

The TechPixel Agent is configured as a **content validator** — its job is to review submitted training notes and determine whether they are genuinely related to Bloxd.io or BloxdHub.com before saving them to the training system.

2. Guidelines Tab

CORE SETUP

This is where the agent's behavior is defined. It contains three fields:

- **Description** — The system prompt defining the agent's role. For TechPixel: *"Validates whether a submitted training note is relevant to Bloxd.io or BloxdHub.com before it is saved."*

- **Instructions** — Detailed rules the agent must follow. This is where the validation logic lives: what topics are on-scope, the JSON output format, strictness level, and rejection criteria.
- **Context Files** — Up to 10 reference files the agent can use. Upload this document as a context file so the agent has a comprehensive knowledge base of all Bloxd.io and BloxdHub topics.

Under **AI Model**, select the model that powers the agent. The default "Automatic" model (~3 credits/message) works for general validation. For more nuanced content analysis, consider Gemini 3 Flash (~5 credits) or GPT-5 (~15 credits).

3. Tools Tab

Controls what data and tools the agent can access. For the TechPixel validator, configure entity tools for reading and creating training_note records. Backend functions can be used to trigger notifications when a note is rejected or auto-tag accepted notes. Adjust permissions so the agent only accesses the training notes entity, not unrelated app data.

4. Skills Tab

Reusable instruction sets attached to the agent. You can create up to 10 skills per agent. For TechPixel, consider creating skills like:

- **bloxd-content-policy** — Detailed rules for what constitutes valid Bloxd.io content.
- **bloxdhub-content-policy** — Rules specific to BloxdHub community content.
- **spam-detection** — Patterns to identify spam, off-topic, or low-value submissions.

Skills can be App Skills (specific to TechPixel) or Workspace Skills (shared across your workspace).

5. Memory Tab

Memory lets the agent remember information across conversations. For TechPixel, enable memory with "Global Only" scope so the agent accumulates knowledge about recurring spam patterns, edge-case rulings, and approved topic areas. Set memory instructions to: *"Save global memories for new Bloxd.io features, API updates, BloxdHub platform changes, and patterns of frequently rejected content types."*

6. Connectors & Channels

Connectors let app users connect external accounts via OAuth. For TechPixel, this is optional but could be used to let users connect Discord for cross-platform moderation. **WhatsApp** and **Telegram** tabs let you connect the agent to messaging apps so moderators can validate content from their phone. Each WhatsApp message costs 1 integration credit.

II. Complete Agent Instructions

1. System Prompt (Description)

Paste this into the **Description** field:

“ Validates whether a submitted training note is relevant to Bloxd.io or BloxdHub.com before it is saved.

2. Full Instructions

PASTE THIS

Paste the following into the **Instructions** field. This is the complete instruction set for the validator agent:

“ You are a strict content validator for the TechPixel Agent training system.
Your only job is to review a submitted training note and determine whether it is genuinely related to Bloxd.io or BloxdHub.com.
Bloxd.io topics include: game mechanics, WorldCode scripting, CodeBlocks, the Bloxd API, world building, mods, items, mobs, events, survival, crafting, combat, scripting, gamemodes, and anything directly related to the Bloxd.io game platform.
BloxdHub.com topics include: the BloxdHub community platform, posts, mods, schematics, API documentation, forums, texture packs, creator tools, community guidelines, and anything related to the BloxdHub social/community site.
Return a JSON object with the following fields:
- relevant: true or false
- reason: a short, clear explanation of your decision (1-2 sentences)
- confidence: "high", "medium", or "low"
If the note is off-topic, vague, spam, unrelated to Bloxd.io or BloxdHub, or does not add useful training value, return relevant: false.
Be strict. When in doubt, return relevant: false.

3. JSON Output Format

The agent must always respond with this JSON structure:

FIELD	TYPE	VALUES	DESCRIPTION
<code>relevant</code>	boolean	true / false	Whether the note is on-topic and adds training value
<code>reason</code>	string	1-2 sentences	Clear explanation of the validation decision
<code>confidence</code>	string	"high" / "medium" / "low"	How certain the agent is about its ruling

III. Bloxd.io — Complete Domain Knowledge

1. Platform Overview

Bloxd.io is a free-to-play, browser-based voxel game platform featuring 30+ gamemodes, custom world building, JavaScript-based scripting, and multiplayer sandbox experiences. It runs at `bloxd.io` and requires no download. The game is developed by a team including Arthur (survival/sandbox modes), Tom (code boards), and Slushie (code editor updates).

The platform has its own scripting API that players can use to create custom game logic via Code Blocks, Code Boards, and World Code. The official API documentation lives at `bloxd.io/docs` and the code-api GitHub repository at `github.com/Bloxdy/code-api`.

2. Complete Gamemodes List

30+ MODES

Any training note referencing these gamemodes is relevant. The full list of current gamemodes:

- **Sandbox/Building:** Worlds (My World), Survival, Peaceful, Creative, One Block, Plots, Sky Islands, Build a Factory
- **PvP/Combat:** Bedwars (Solos, Duos, Trios, Squads, 4v4), Skywars, Cube Warfare (Free-For-All, Teams), Last Stand, 1v1 Fights (Sword Fight, Build Fight, Bridge Fight, Bed Fight), Core Defenders, Survival with Guns, Clutch
- **Minigames:** Greenville (Isle of Grove, Spleef), Doodle Cube, Infection, Pirates, Lucky Blocks Battlegrounds, Lucky Towers, Murder Mystery, Hide and Seek, Evil Tower, Tsunami Steal Brainrots
- **Social/Other:** Fishing, Custom Games, The Bloxd SMP, 99 Nights

Notes about gamemodes that have been removed or archived are also valid training content as historical reference.

3. Survival Mode Mechanics

Survival is the most popular sandbox gamemode. Key mechanics the agent should recognize as valid:

- **Commands:** /sethome (up to 3 homes), /tprequest (teleport requests), Teleport to lobby spawn
- **Tribes:** Created for 2 iron bars via B-menu. Features include Set Tribe Home, Invite/Remove players, Toggle PvP, Disband Tribe, and Tribe Protectors
- **Protectors:** Regular protectors (1 messy stone + 2 diamonds, up to 8) and Tribe Protectors (1 messy stone + 2 moonstones, up to 4). Protect chunks from editing by other players. Disappear after 30 days of inactivity.
- **Combat Logging:** Players below 80 HP who leave within 10 seconds of being attacked are automatically killed and items dropped
- **Spawn Reset:** Blocks within 192 blocks of spawn reset every 150 minutes, allowing ore and loot chest farming
- **Crafting, mining, farming, mobs, items, ores, loot chests, beds, spawn points**

4. Code Blocks & Code Boards

SCRIPTING

Code Blocks are physical blocks placed in the world that execute custom JavaScript when a player clicks them or an adjacent "press to code" board. They are found in the creative menu.

- Code runs in a JavaScript sandbox using QuickJS (no `window`, `setTimeout`, or browser APIs)
- 16,000 character limit per Code Block
- Global variables: `myId` / `playerId` (player who clicked), `thisPos` (block position [x,y,z])
- Code resets every time the block is used (no persistent state unless using global variables)
- Only world owners, co-owners, and coders (users with `canEditCode = true`) can place or edit them
- `async` functions cannot be used (code runs synchronously)
- Avoid `let`, `const`, or `var` in global scope; wrap in IIFEs

Code Boards are signs that begin with "press to code" to run JavaScript on right-click. They have very limited text space (16×6 = 96 characters). Use Code Blocks for anything longer.

5. World Code SCRIPTING

World Code is the main way to write persistent game logic. Accessed by pressing F8, it runs when callbacks are triggered. Only world owners can edit it. 16,000 character limit.

Key differences from Code Blocks: World Code supports **callbacks** (event-driven), `myId` and `thisPos` are `null`, and it executes on lobby load or when updated.

All 64 callbacks the agent should recognize as valid topics:

- **Lifecycle:** tick, onClose, onPlayerJoin, onPlayerLeave, onRespawnRequest
- **Movement:** onPlayerJump, onBlockStand
- **Chat/Commands:** onPlayerChat, playerCommand
- **Building:** onPlayerChangeBlock, onWorldChangeBlock, onChunkLoaded, onPlayerRequestChunk
- **Inventory:** onPlayerDropItem, onPlayerPickedUpItem, onPlayerSelectInventorySlot, onPlayerMoveItemOutOfInventory, onPlayerMoveInvenItem, onPlayerMoveItemIntoDxs, onPlayerSwapInvenSlots, onPlayerMoveInvenItemWithAmt, onInventoryUpdated, onItemDropCreated
- **Crafting/Chests:** onPlayerAttemptCraft, onPlayerCraft, onPlayerAttemptOpenChest, onPlayerOpenedChest, onChestUpdated
- **Combat:** onPlayerAttack, onPlayerDamagingOtherPlayer, onPlayerDamagingMob, onMobDamagingPlayer, onMobDamagingOtherMob, onAttemptKillPlayer, onPlayerKilledOtherPlayer, onMobKilledPlayer, onPlayerKilledMob, onMobKilledOtherMob, onPlayerPotionEffect
- **Mob management:** onPlayerAttemptSpawnMob, onWorldAttemptSpawnMob, onPlayerSpawnMob, onWorldSpawnMob, onWorldAttemptDespawnMob, onMobDespawned
- **Interaction:** onPlayerAttemptAltAction, onPlayerAltAction, onPlayerClick, onPlayerClickUp, onTouchscreenActionButton
- **Mesh entities:** onCreateBloxdMeshEntity, onEntityCollision, onPlayerDamagingMeshEntity, onPlayerBreakMeshEntity
- **Throwables:** onPlayerUsedThrowable, onPlayerThrowableHitTerrain
- **Items:** onPlayerStartChargingItem, onPlayerFinishChargingItem
- **Settings:** onClientOptionUpdated, onMobSettingUpdated
- **Shop/Tasks:** onPlayerBoughtShopItem, onTaskClaimed
- **QTE:** onPlayerFinishQTE
- **Persistence:** doPeriodicSave

6. Bloxd API Reference

CORE API

The `api` object is the global interface for all game interactions. Any training note referencing these API methods is valid. Organized by category:

Player Actions:

- `api.setVelocity(playerId, x, y, z)` — Set player velocity
- `api.applyImpulse(playerId, x, y, z)` — Push the player
- `api.setCameraDirection(playerId, [x, y, z])` — Control camera
- `api.forceRespawn(playerId)` — Force respawn
- `api.setPlayerPose(playerId, "zombie")` — Set pose (zombie, driving, etc.)
- `api.setPlayerOpacity(playerId, value)` — Control visibility
- `api.kickPlayer(playerId, "reason")` — Kick player

Messaging:

- `api.sendMessage(playerId, "text", {color: "cyan"})` — Send to one player
- `api.broadcastMessage("text", {color: "red"})` — Send to all
- `api.sendFlyingMiddleMessage(playerId, ["text"], speed)` — Flying text
- `api.sendTopRightHelper(playerId, "icon", "text", options)` — Top-right helper

Entity & Block Control:

- `api.getPosition(entityId)` / `api.setPosition(entityId, x, y, z)`
- `api.getPlayerIds(limit)` / `api.playerIsInGame(playerId)` / `api.getNumPlayers(limit)`
- `api.getBlockCoordinatesPlayerStandingOn(playerId)` / `api.getBlockTypesPlayerStandingOn(playerId)`
- `api.setBlock(x, y, z, "Block Name")` / `api.getBlock(x, y, z)` / `api.setBlockRect(pos1, pos2, "Block Name")`

Health & Damage:

- `api.getHealth(entityId)` / `api.setHealth(entityId, value)` / `api.applyHealthChange(entityId, amount)`
- `api.killLifeform(entityId)` / `api.isAlive(entityId)`
- `api.getCurrentKillstreak(playerId)` / `api.clearKillstreak(playerId)`

Inventory & Items:

- `api.giveItem(playerId, "ItemName", amount, options)` — With optional `customDisplayName`, `customAttributes`, `enchancements`
- `api.removeItemName(playerId, "ItemName", amount)`
- `api.getHeldItem(playerId)` / `api.setItemSlot(playerId, slot, "ItemName")` / `api.clearInventory(playerId)`

Chest & Crafting:

- `api.getStandardChestItems([x,y,z])` / `api.setStandardChestItemSlot([x,y,z], slot, "ItemName")`
- `api.editItemCraftingRecipes(playerId, "ItemName", recipes)` / `api.resetItemCraftingRecipes(playerId, "ItemName")`

Mesh Attachments:

- `api.updateEntityNodeMeshAttachment(entityId, node, type, options, position, rotation)`
- `api.scalePlayerMeshNodes(playerId, nodeScalingMap)`

- Nodes: TorsoNode, HeadMesh, ArmRightMesh, ArmLeftMesh, LegLeftMesh, LegRightMesh

Mob Spawning:

- `api.attemptSpawnMob("MobType", x, y, z, options)` — Options include name, variation, playSoundOnSpawn, physicsOpts
- Mob variations: "iron", "default", "black", "red", "orange", "pink", "purple", "yellow", "blue", "brown", "cyan", "gray", "green", etc.

Skins & Appearance:

- `api.changePlayerIntoSkin(playerId, "head", skinName)` — Skins: farmer, Wizard, Zombie, portal_mage, trader
- `api.setTargetedPlayerSettingForEveryone(playerId, "nameTagInfo", options)`

Effects:

- `api.applyEffect(playerId, "EffectName", duration, {inbuiltLevel: N})`
- Effects include: Damage, Damage Reduction, Speed, Frozen, Invisible, Jump Boost, Poisoned, Slowness, Weakness, Health Regen, Haste, Double Jump, Blindness, Lifesteal, Wall Climbing, Thorns, Air Walk, Bounciness, and many more

UI & Visuals:

- `api.playParticleEffect(options)` — Textures: bubble, critical_hit, drift, effect_5, generic_2, glint, heart, soul_0, square_particle
- `api.showShopTutorial(playerId)`
- `api.progressBarUpdate(playerId, progress, duration)`
- `api.createShopItem(category, itemKey, options)`

Client Options:

- `api.setClientOption(playerId, "airJumpCount", N)` — Double/multi jump
- `api.setClientOption(playerId, "music", "songName")` — Custom music

QTEs (Quick Time Events):

- `api.addQTE(playerId, options)` — Types include "progressBar" with parameters: progressStartValue, progressDecreasePerTick, progressPerClick, canFail, description, clickIcon, scale, rotation

Utilities:

- `api.log("message")` / `console.log("message")` — Debugging
- `api.now()` / `Date.now()` — Timestamps in milliseconds
- `api.checkValid(entityId)` — Validate entity existence

7. Music & Sounds

Bloxd.io has a built-in music system accessible via

`api.setClientOption`. Valid song names include:

- Adigold tracks: A Place To Be Free, Butterfly Effect, Dreamless Sleep, Frozen Pulse, Frozen Skies, Healing Thoughts, Here Forever, Just a Little Hope, Just Like Heaven, Memories Remain, Place To Be, The Riverside, The Wonder, Vetrar (Cut B)
- Retro tracks: Juhani Junkala [Retro Game Music Pack] (Ending, Level 1, Level 2, Level 3, Title Screen)
- Other: cdk-Silence-Await, battle-ship-111902, ghost-Reverie-small-theme, happy, Heroic-Demise-New, TownTheme, progress, and more

8. Items, Blocks & Enchantments

Training notes about any of these topics are valid:

- **Ores & Materials:** Iron, Diamond, Moonstone, Messy Stone, Gold Coin, Coal, Wheat, Corn, Cotton, Stick, etc.
- **Tools & Weapons:** Diamond Sword, Diamond Spear, Diamond Boomerang, Diamond Dagger, Diamond Whip, Iron Sword, pickaxes, axes, etc.
- **Enchantments:** Vertical Knockback, Critical Damage, and enchantment tiers (Tier 1-5). Custom enchantments via `customAttributes`
- **Blocks:** The full block list is available at `github.com/Quentin-X/filehosting/blob/main/blocks.js`. Includes Stone, Dirt, Gold Block, Moonstone Block, Red Chalk, Red Carpet, and hundreds more
- **Food:** Bread, Cornbread, Apple, and crafted food items

9. Type System & Advanced Concepts

Advanced scripting types the agent should recognize as valid topics:

- **CustomTextStyling** — Arrays of styled text, entity names, icons, and translated text for rich UI
- **EntityMeshScalingMap** — Per-node mesh scaling for character customization
- **TempParticleSystemOpts** — Full particle system configuration including textures, gradients, gravity, blend modes
- **ShopItem / ShopCategoryConfig** — In-game shop configuration with costs, currencies, user inputs (text, number, dropdown, player, color)
- **MobSpawnOpts** — Mob spawning with herds, variations, physics options
- **MeshEntityOpts** — Types include Box, BloxdBlock, Person, ParticleEmitter
- **AnimationSchema / BlockbenchAnimationSchema** — Custom animations with keyframes, lerp modes, catmull-rom splines
- **RecipesForItem** — Custom crafting recipes with stations, aura effects, starter recipe flags

- ParticleSystemBlendMode — OneOne (0), Standard (1), Add (2), Multiply (3), MultiplyAdd (4)

10. Code Examples

Training notes containing code examples like these are valid:

- Jump pad: `api.setVelocity(myId, 0, 12, 0)`
- Give item: `api.giveItem(myId, "Diamond", 1)`
- Trampoline via World Code: `onBlockStand=(id,x,y,z,blockName)=>{
if(blockName=="Red Chalk") api.setVelocity(id,0,20,0) }`
- Welcome message: `function onPlayerJoin(id) {
api.sendTopRightHelper(id, "star", "WELCOME", {color:"yellow"}) }`
- Freeze on hit: Using onPlayerDamagingOtherPlayer callback with `api.applyEffect(pid,"Frozen",5000,{inbuiltLevel:1})`
- Shop systems: Using `api.createShopItem` with categories, costs, and buy handlers
- Particle effects: Smoke bombs, bubble effects using `api.playParticleEffect` with gradients and blend modes
- Custom skins/capes: Using mesh attachments to create capes, karts, golem figures

IV. BloxdHub.com — Complete Domain Knowledge

1. Platform Overview

BloxdHub.com is the community social platform for Bloxd.io players. It is NOT affiliated with Bloxd Ltd. It features posts, forums, mods, schematics, texture packs, speedruns, Bloxdboards, and comprehensive Bloxd API documentation. Users sign in via Google or Discord. The platform has ~19,400 total users.

Navigation sections: Home, Bloxdboards, Forums, Mods, Schematics, Speedruns, Texture Packs, Bloxd API Docs.

2. Content Sections

7 SECTIONS

Each of these sections and their content are valid training topics:

- **Posts / Home Feed:** User posts with tags (Builds, Memes, Screenshots, Clips, Questions, Tips, Suggestions, Announcements, Updates, Showcases, Guides, Discussions, Feedback, Bugs, Lore, Reports, Advertisements, Off Topic, Artwork, Rants). Feed modes: Recent, Following, Smart, Custom, Top posts this week.
- **Bloxdboards:** Community boards with voting, similar to Reddit. Users can post and interact on trending boards.
- **Forums:** Dedicated discussion areas for coding help, general discussions, guides, and Q&A.
- **Mods:** User-uploaded game modifications with descriptions, instructions, and preview images.
- **Schematics:** Shared building schematics with instructions and required tools.

- **Speedruns:** Competitive speedrun submissions and leaderboards for various gamemodes.
- **Texture Packs (TPacks):** Custom texture packs that modify the visual appearance of the game.

3. Community Guidelines

BloxdHub enforces 25 community rules (BH-R1 through BH-R25).

Training notes about any of these are valid:

- BH-R1 through BH-R5: Respect, no harassment, safe content, no NSFW, no spam
- BH-R6 through BH-R10: Correct sections, credit others, no personal info, no trolling, constructive feedback
- BH-R11 through BH-R15: No unauthorized advertising, follow formatting, no exploits, no impersonation, appropriate usernames
- BH-R16 through BH-R20: Respect mods, report instead of fight, no begging, no excessive caps/emoji, no hate
- BH-R21 through BH-R25: Proper forum use, relevant discussions, no piracy, no misinformation, follow updates

Rule violations result in warnings, temporary bans, or permanent suspension.

4. BloxdHub API Documentation

BloxdHub hosts comprehensive community-written API docs at

bloxdhub.com/bloxd-api. This is not official documentation but an additional community resource. Currently covers approximately 45-50% of the Bloxd API. Sections include:

- **Getting Started:** Introduction, Your First Script, Syntax Basics
- **Core Concepts:** Events (callbacks)
- **Players:** Messages, Player Info, Position & Movement, Camera & View, Appearance, Health & Combat, Inventory
- **World & Blocks:** Basic Blocks, Block Areas, Block Permissions
- **Entities & Mobs:** Spawning Mobs, Mob Settings
- **Client Settings:** Overview & Setup, Movement & Physics, Combat & Health, UI & Display, World & Building
- **Upcoming sections:** Particles, Sounds & Music, UI Elements, Damage & Shields, Status Effects, Lobby & Utilities, Crafting & Recipes, Enchanting

5. User Features & Engagement

Features the agent should recognize as valid BloxdHub topics:

- **Profiles:** Usernames (max 16 chars), bios (max 150 chars), social links (up to 5), dynamic banners, avatar
- **Engagement Streak:** Daily activity tracking (post, comment, or react) with current and best streaks
- **Boosts:** Rare points earned by activity. Used for advertising mods/worlds, highlighting posts, profile themes/badges, donations, exclusive events
- **Creator Insights:** Analytics for optimizing posting strategy
- **Collab Posts:** Invite one collaborator to co-create a post (v0.55 feature)
- **Reactions:** 20+ reactions including standard emoji and custom ones (crypray, bigthonk, thinkly, cutie)
- **Notifications:** Configurable for mentions, comments, replies, followers, pinned comments, likes, reactions

- **Themes:** Default, Bloxd Classic, Bloxd Underground, Icy Lake, Capture the Flag, House, Cinematic Sunset, Rainy, Crimson Night, Electric Yellow, Midnight Rose
- **Privacy settings:** Hide profile tabs (Reactions, Comments, Mods, Schematics, Boards)

6. YouTuber Program

BloxdHub runs a YouTuber verification program. Requirements:

- Minimum 500 subscribers and Bloxd-related content
- Option 1: 1 video/month with 20+ seconds of BloxdHub and 750+ views
- Option 2: 2 videos/month with 20+ seconds each and 400+ views each
- Dedicated Video: One BloxdHub video with 1,500+ views = 2 months qualification
- Benefits: Special name color, badge, ability to post YouTube links

7. Related Tools & Ecosystem

External tools and resources in the Bloxd ecosystem are also valid topics:

- **BloxdForge (bloxdforge.com):** Studio Hub (Launcher, Workshop), Texture Studio, Remix Studio, World Tools, and a wiki with comprehensive API docs
- **Bloxd.io Wiki (bloxd-io.fandom.com):** Community-maintained Fandom wiki with detailed pages on every gamemode, item, mob, and mechanic
- **GitHub (github.com/Bloxdy):** Official code-api repository with README, API_REFERENCE, CALLBACKS, CLIENT_OPTIONS, ENTITY_SETTINGS, ICONS, MESH_ENTITY_DOCS, MOB_SETTINGS, PARTICLES, QTE_DOCS, SKINS_AND_POSES, SOUNDS_AND_MUSIC, BLOCK_NAMES, ITEM_NAMES
- **Discord:** Official Bloxd community Discord server for bug reports, feature requests, and community chat

V. Validation Examples & Edge Cases

1. ACCEPT — High Confidence

These training notes should always be accepted:

RELEVANT: TRUE

"In Survival mode, protectors require 1 messy stone and 2 diamonds. Players can place up to 8 protectors."	<pre>{"relevant": true, "reason": "Accurately describes Survival protector mechanics including crafting recipe and placement limits.", "confidence": "high"}</pre>
"Use api.setVelocity(myId, 0, 12, 0) to create a jump pad in Code Blocks."	<pre>{"relevant": true, "reason": "Valid Code Block example using the Bloxd.io scripting API for player movement.", "confidence": "high"}</pre>
"BloxdHub community rule BH-R6 requires users to post content in the correct sections (mods, texture packs, schematics)."	<pre>{"relevant": true, "reason": "Accurately references a specific BloxdHub community guideline.", "confidence": "high"}</pre>
"The onPlayerKilledOtherPlayer callback fires when one player kills another and can be used to create coin-drop mechanics."	<pre>{"relevant": true, "reason": "Describes a valid World Code callback with a practical use case.", "confidence": "high"}</pre>
"BloxdHub's Bloxdboard section lets users create community boards with voting, similar to Reddit."	<pre>{"relevant": true, "reason": "Describes a core BloxdHub.com feature.", "confidence": "high"}</pre>
"Bedwars in Bloxd.io supports Solos, Duos, Trios, Squads, and 4v4 modes."	<pre>{"relevant": true, "reason": "Accurately lists the sub- modes of a major Bloxd.io gamemode.", "confidence": "high"}</pre>

2. ACCEPT — Medium Confidence

RELEVANT: TRUE

These notes are valid but may have minor issues:

TRAINING NOTE	EXPECTED OUTPUT
"Bloxd uses QuickJS for its scripting engine."	{"relevant": true, "reason": "Relates to the technical architecture of Bloxd.io scripting, though lacks detail.", "confidence": "medium"}
"The Bloxd API supports custom particle effects with multiple texture types."	{"relevant": true, "reason": "Discusses a valid API feature, though not specific enough to be highly useful.", "confidence": "medium"}
"BloxdHub recently added collab posts in version 0.55."	{"relevant": true, "reason": "References a real BloxdHub feature update.", "confidence": "medium"}

3. REJECT — High Confidence

RELEVANT: FALSE

These training notes should always be rejected:

TRAINING NOTE	EXPECTED OUTPUT
"Minecraft has a new update with cherry blossom biomes."	{"relevant": false, "reason": "About Minecraft, not Bloxd.io. Different game platform entirely.", "confidence": "high"}
"How to make a website with React and Next.js."	{"relevant": false, "reason": "General web development topic unrelated to Bloxd.io or BloxdHub.", "confidence": "high"}
"Buy cheap V-Bucks at this link."	{"relevant": false, "reason": "Spam/advertising for a different game (Fortnite). No Bloxd.io connection.", "confidence": "high"}
"The weather is nice today."	{"relevant": false, "reason": "Completely off-topic. No relation to Bloxd.io or BloxdHub.", "confidence": "high"}
"Roblox Studio scripting tutorial for beginners."	{"relevant": false, "reason": "About Roblox, not Bloxd.io. Different platform despite some similarities.", "confidence": "high"}
"Check out my YouTube channel for gaming content!"	{"relevant": false, "reason": "Generic self-promotion without Bloxd.io or BloxdHub context.", "confidence": "high"}

4. REJECT — Edge Cases

RELEVANT: FALSE

Borderline cases that should still be rejected:

TRAINING NOTE	EXPECTED OUTPUT
"JavaScript is a programming language."	<pre>{"relevant": false, "reason": "Generic programming fact. While Bloxd uses JavaScript, this note doesn't specifically relate to Bloxd scripting.", "confidence": "high"}</pre>
"Voxel games are really popular right now."	<pre>{"relevant": false, "reason": "Generic observation about voxel games without specific Bloxd.io content.", "confidence": "medium"}</pre>
"I love playing games online."	<pre>{"relevant": false, "reason": "Vague statement with no specific Bloxd.io or BloxdHub reference.", "confidence": "high"}</pre>
"Multiplayer sandbox games should have better moderation."	<pre>{"relevant": false, "reason": "Generic opinion about sandbox games, not specifically about Bloxd.io or BloxdHub.", "confidence": "medium"}</pre>
"api.setVelocity is a function."	<pre>{"relevant": false, "reason": "Too vague and low-value despite referencing a real API method. Doesn't add useful training context.", "confidence": "medium"}</pre>

5. Decision Flowchart

Follow this logic for every submitted note:

- **Step 1:** Does the note mention Bloxd.io, BloxdHub, or any specific feature, gamemode, API method, or community section of either platform? → If NO, reject immediately with high confidence.
- **Step 2:** Is the note spam, self-promotion, or an advertisement unrelated to either platform? → If YES, reject with high confidence.
- **Step 3:** Does the note contain factual, useful, or instructional content about either platform? → If YES, accept. If the note is vague but on-topic, accept with medium confidence.
- **Step 4:** Is the content about a different game (Minecraft, Roblox, Fortnite, etc.) even if the game has similar mechanics? → Reject with high confidence.
- **Step 5:** Does the note mention general programming, web development, or technology without specific Bloxd.io/BloxdHub context? → Reject. Generic JavaScript knowledge is NOT sufficient unless applied to the Bloxd API.
- **Step 6:** When in doubt, err on the side of rejection. The agent should be strict.

VI. Complete Topic Taxonomy

1. Bloxd.io Valid Topics

Any training note about these topics should be accepted:

ACCEPT

CATEGORY	SPECIFIC TOPICS
Game Mechanics	Survival mechanics, tribes, protectors, combat logging, spawn resets, commands (/sethome, /tprequest), crafting, mining, farming, mobs, items, ores, loot
Gamemodes	All 30+ gamemodes including Survival, Bedwars, Skywars, Creative, Worlds, One Block, Peaceful, Plots, Doodle Cube, Cube Warfare, Last Stand, 99 Nights, Infection, Pirates, Lucky Blocks, Murder Mystery, Hide and Seek, Evil Tower, 1v1 Fights, Custom Games, Fishing, Greenville, etc.
WorldCode Scripting	F8 editor, callbacks (all 64), event-driven logic, World Code vs Code Blocks, global variables, debugging with api.log
CodeBlocks	Code Block placement, execution model, QuickJS environment, 16K char limit, code boards, "press to code", IIFEs, sandbox limitations
Bloxd API	All api.* methods: player actions, messaging, entity/block control, health/damage, mesh attachments, inventory, chests, crafting, UI/visuals, particles, effects, client options, QTEs, shops, utilities
World Building	Creative mode building, block placement, schematics, terrain, chunk mechanics, world settings
Mods	Game modifications, custom gamemodes, mod creation, mod sharing
Items & Blocks	All item names, block names, crafting recipes, enchantments, custom items via customAttributes
Mobs & Entities	Mob types, mob variations, mob spawning, mob settings, mesh entities, entity collision
Events & Effects	All status effects (Frozen, Poisoned, Speed, etc.), particle textures, sound/music system
Combat	PvP mechanics, damage systems, killstreaks, health shields, weapon types, armor
Developer Info	Arthur, Tom, Slushie (developers), update history, patch notes, feature additions
Technical	QuickJS runtime, animation schemas (native + Blockbench), blend modes, lerp modes, type system

2. BloxdHub.com Valid Topics

ACCEPT

CATEGORY	SPECIFIC TOPICS
Platform Features	Home feed, post tags (20+ types), feed modes (Recent, Following, Smart, Custom), collab posts, engagement streaks, boosts, creator insights
Posts & Content	Posting rules, content categories, series, GIFs, image uploads, reactions (20+ types including custom), comments, pinning
Mods Section	Mod uploads, descriptions, instructions, preview images, credits, testing
Schematics	Schematic sharing, instructions, required tools, credit attribution
Texture Packs	TPack uploads, custom textures, visual modifications
Forums	Coding help, general discussions, guides, Q&A
Bloxdboards	Community boards, voting, trending boards
Speedruns	Speedrun submissions, leaderboards, gamemode-specific runs
API Documentation	BloxdHub's community-written API docs (all sections listed in IV.4)
Community Rules	All 25 community guidelines (BH-R1 through BH-R25)
User Features	Profiles, usernames, bios, social links, avatars, banners, themes (11 themes), privacy settings, notification settings
Programs	YouTuber program, moderator selection, Discord integration
Moderation	Reporting system, spam categories, content moderation, ban system

3. Always REJECT Topics

REJECT

CATEGORY	WHY REJECT
Other games (Minecraft, Roblox, Fortnite, etc.)	Different platforms entirely, even if mechanics are similar
General programming (HTML, CSS, React, Python)	Not Bloxd-specific even if the language overlaps
Generic JavaScript tutorials	Must be applied to the Bloxd API specifically
Personal opinions with no factual content	No training value added
Spam, advertisements, self-promotion	No relation to either platform
Off-topic discussions (weather, school, food)	Completely unrelated
Other game platforms (Steam, Epic, itch.io)	Different ecosystems
AI/LLM discussions not about Bloxd agents	Generic tech topic
Social media platforms (Discord, YouTube) in general	Only valid if specifically about Bloxd Discord or BloxdHub YouTuber program

VII. Base44 Setup Checklist

Step-by-Step Setup

Follow this exact sequence to configure the TechPixel agent in Base44:

- 1. **Create the Agent:** Go to Dashboard → Agents → Create from scratch. Name it `training_note_content_validator`.
- 2. **Set the Description:** Paste: "Validates whether a submitted training note is relevant to Bloxd.io or BloxdHub.com before it is saved."
- 3. **Set the Instructions:** Paste the full instruction block from Section II.2 of this document.
- 4. **Upload Context Files:** Export this training document as a text file and upload it as a Context File (0/10 files → 1/10). This gives the agent comprehensive domain knowledge.
- 5. **Select AI Model:** Start with "Automatic" (~3 credits/message). Upgrade to Gemini 3 Flash (~5 credits) if the agent struggles with edge cases.
- 6. **Configure Tools:** Enable read access to the `training_notes` entity. Enable create/update permissions only if the agent should auto-save validated notes.
- 7. **Create Skills:** Optionally create `bloxd-content-policy` and `spam-detection` App Skills with specialized instructions.
- 8. **Enable Memory:** Turn on memory with "Global Only" scope. Set instructions to remember new Bloxd features and spam patterns.
- 9. **Test:** Submit test notes from each category (accept high, accept medium, reject high, reject edge cases) and verify JSON output format.
- 10. **Deploy:** Connect WhatsApp or Telegram if moderators need mobile access. Click Save.

JSONC Config (Developer)

If using the Base44 developer CLI, create this file at

`base44/agents/training_note_content_validator.jsonc`:

```
“ {
  "name": "training_note_content_validator",
  "description": "Validates whether a submitted training note
is relevant to Bloxd.io or BloxdHub.com before it is saved.",
  "instructions": "[Full instructions from Section II.2]",
  "model": "anthropic/claude-sonnet-4-20250514",
  "tool_configs": [
    { "entity_name": "training_notes", "allowed_operations":
["read", "update"] }
  ]
}
```

Run `agents push` to deploy.

VIII. Maintenance & Ongoing Updates

Keeping the Agent Current

Both Bloxd.io and BloxdHub evolve regularly. To keep the agent's validation accurate:

- **New Gamemodes:** When Bloxd.io adds a new gamemode, update the context file to include it in the valid topics list. Check the Bloxd.io Wiki and official Discord for announcements.
- **API Changes:** Monitor the github.com/Bloxdy/code-api repository for new commits. Tom (tom288) maintains the API reference. Last updated May 13, 2026.
- **BloxdHub Updates:** Check the "What's New" section on BloxdHub. Current version is v0.55 (April 18, 2026). New features like Collab Posts should be added to the valid topics.
- **Community Guidelines:** If BloxdHub updates its rules (currently BH-R1 through BH-R25), update the context file accordingly.
- **Memory Review:** Periodically review the agent's saved memories in the Memory tab. Remove outdated or incorrect memories. The agent should accumulate knowledge about edge-case rulings over time.
- **Model Upgrades:** As newer AI models become available, test whether upgrading improves edge-case accuracy. The "Automatic" model may be sufficient for straightforward validation.

